EDITION 2026 · ADVANCED ENGINEERING TRACK

DevSecOps Lab Book

Zero to Hero

Master Plan & Full Architecture Reference — a from-scratch build of a production-grade DevSecOps platform, one VM, one certificate, one firewall rule at a time.

28	14	7	7
CHAPTERS	LAB VMS	PHASES	SECURITY LAYERS

Who This Book Is For

> You don't need to be a security expert to start. You need to be curious, patient, and willing to understand *why* before *how*. By the last chapter, you will have built a production-grade DevSecOps platform from scratch — every service, every certificate, every firewall rule.

Every command is explained: not just what to run, but why it exists and what breaks if you skip it.

28

Chapters across 7 phases, from DNS to SIEM

14

Purpose-built lab VMs, ~890 GB total disk

7

Defense-in-depth security layers, wall to wall

7

Security-series parts per major tool

COURSE MAP

Seven Phases, Twenty-Eight Chapters

Each phase builds on the last — from bare-metal foundations to a fully observed, self-defending platform.

01



Foundation

Ch 01–05

DNS · Identity

02



Immutable Infra

Ch 06–08

Packer · Tofu · Ansible

03



Security Core

Ch 09–13

Proxy · Vault · Teleport

04



CI/CD Supply Chain

Ch 14–19

GitLab · Sonar · Harbor

05



Orchestration

Ch 20–22

Kubernetes · GitOps

06



Observability & SIEM

Ch 23–24

ELK · Wazuh

07



Cross-Cutting Controls

Ch 25–28

iptables · AppArmor · Auditd

LAB ENVIRONMENT

The Complete VM Inventory

Fourteen purpose-built VMs, grouped by security domain — plus the hypervisor host.

Domain	Representative Hosts	vCPU	RAM	Disk
Foundation	sec-dns-01 · sec-idm-01	2	8 GB	60 GB
Security Core	sec-edge-01 · sec-vault-01	8	8 GB	70 GB
CI/CD Supply Chain	sec-vcs-01 · sec-appsec-01 · sec-registry-01	16	24 GB	240 GB
Orchestration	k8s-cp-01/02/03 · k8s-worker-01/02	20	34 GB	340 GB
Observability & SIEM	sec-obs-01 · sec-siem-01	10	20 GB	200 GB

MINIMUM LAB FOOTPRINT

56

VCPU CORES

94 GB

RAM ALLOCATED

~890 GB

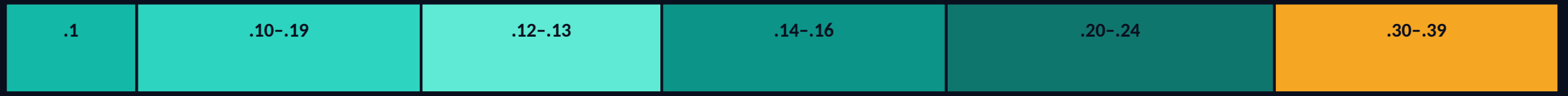
DISK PROVISIONED

Resource-constrained? Run phases sequentially — snapshot, shut down, spin up the next. A 32 GB / 8-core host completes every phase this way.

NETWORK DESIGN

Network & IP Architecture

10.0.2.0/24 — GATEWAY 10.0.2.1



Host / Gateway	Foundation	Security Core*	CI/CD Supply Chain	Kubernetes Cluster	Observability & SIEM

** Overlaps foundation range intentionally — the edge and vault tier is treated as core security infrastructure.*

Domain Naming Convention

Internal domain: **devsecops.com**
Internal zone: **internal.devsecops.com**

Service CNAMEs resolved by BIND9: gitlab · vault · harbor · sonar · auth · kibana · wazuh → each mapped to its host VM.

Why Split-Brain DNS

Internal clients resolve service names to private IPs; external queries never see internal topology.
BIND9 answers differently depending on which network asked — the same name, two truths, one authority.

SYSTEM DESIGN

Full Architecture at a Glance



HOW TRAFFIC MOVES

- Developer workstation reaches everything through the Edge Proxy — the single entry point.
- Edge validates every session against Authentik (OIDC) before forwarding traffic anywhere.
- Vault issues PKI certificates to Edge and Identity, and injects runtime secrets into GitLab.
- GitLab pushes images to Harbor and triggers scans in SonarQube — before Kubernetes ever sees them.
- Every host ships logs to ELK and a HIDS agent to Wazuh — DNS resolves the whole map.

END-TO-END FLOW

A Developer Pushes Code

Every gate this pipeline enforces exists because something, somewhere, once shipped without it.

01

git push

over HTTPS/SSH to the Edge

02

Validate

OIDC token checked by Authentik

03

CI Pipeline

GitLab runs the pipeline

04

SAST Scan

SonarQube quality gate

05

Build Image

Docker image assembled

06

CVE Scan

Trivy scans the container

07

Sign & Push

Harbor verifies with Cosign

08

Deploy

Kubernetes — only if all gates pass

DEFENSE IN DEPTH

The Seven Security Layers

Every tool in this lab maps to at least one layer. A mature platform has controls at every level.



HOW TO READ THIS BOOK

Every Chapter Follows One Format

1. Why This Tool Exists

The problem it solves

2. Core Concepts

Mental models, analogies

3. Architecture in Our Lab

How it fits our specific design

4. Installation

Step-by-step, with explanations

5. Configuration

Every important setting explained

6. Integration

How it talks to other tools

7. Validation & Testing

Prove it works

8. Troubleshooting

Real errors, real fixes

9. Exercises

Hands-on labs with solutions

10. Security Series

7-part deep security hardening

DEPTH OVER BREADTH

The Security Series — 7 Parts Per Tool

Every major tool in the book gets this same hardening treatment, not just an install guide.

01	**Threat Model**	*What can go wrong with this tool?*
02	**Authentication Hardening**	*Locking down who can log in*
03	**Authorization & Least Privilege**	*Limiting what they can do once in*
04	**Encryption in Transit & at Rest**	*Protecting data on the wire and on disk*
05	**Audit Logging & Alerting**	*Knowing when something happens*
06	**Backup, Recovery & Continuity**	*Surviving the worst day*
07	**Compliance Mapping**	*CIS / NIST / SOC2 traceability*

PHASE OVERVIEW · 1 OF 2

Phases 1–4: Building the Base

1 Foundation

Goal: A working network, DNS, and identity platform — every VM resolves names, every service has a user directory.

Why now: *Nothing else works without DNS and identity. These are the bedrock.*

2 Immutable Infrastructure

Goal: Reproducible, auditable VM creation — golden images, IaC, idempotent configuration.

Why now: *You need to be able to rebuild the entire lab from code.*

3 Security & Access Core

Goal: All traffic flows through authenticated, encrypted proxies. Secrets are managed.

Why now: *Before connecting services together, secure the connections.*

4 CI/CD Supply Chain

Goal: A pipeline that enforces security gates before anything reaches production.

Why now: *The CI/CD pipeline is the highest-value attack surface.*

Phases 5–7: Running It, Watching It, Locking It Down

5 Orchestration

Goal: Workloads run in a secure, policy-enforced Kubernetes cluster — RBAC, NetworkPolicy, PodSecurityStandards, GitOps.

Why now: *Kubernetes security builds on everything before it.*

6 Observability & SIEM

Goal: See everything that happens, alert on anything suspicious — centralized logs, dashboards, HIDS alerts, playbooks.

Why now: *Detection requires full visibility across all systems.*

7 Cross-Cutting Controls

Goal: Host-level hardening applied consistently across all VMs — firewall rules, MAC, time sync, audit trails.

Why now: *Applied to every machine as it's provisioned; this phase documents them as a standalone reference.*

READING THIS BOOK

Conventions Used Throughout



`$ command`

Run on the HOST (your laptop/desktop)



`vm# command`

Run INSIDE a VM



`vm-root# command`

Run as root inside a VM



`// file.yaml`

File contents — filename appears above



`→ output`

Expected output from a command



⚠️ `Warning`

This will break X if you skip Y



🔐 `Security note`

This setting prevents credential exposure



💡 `Key takeaway`

The one thing to remember from this section

READY TO BUILD?

Next: Chapter 01

Host Preparation & Hypervisor Setup

Curious. Patient. Willing to understand why before how. That's all this chapter asks of you.